

Project Documentation: Regional Retail Intelligence Pipeline

Role: BI Data Analyst

Objective: Build an end-to-end data pipeline to analyze 1,000+ "dirty" retail transactions across Singapore, Thailand, and Dubai.

Source: [Kaggle](#).

About Dataset: The Dirty Retail Store Sales dataset contains 12,575 rows of synthetic data representing sales transactions from a retail store. The dataset includes eight product categories with 25 items per category, each having static prices.

Phase 1: Data Ingestion & SQL Staging

Tool: MySQL Workbench

Step 1) Creating database, staging and production database.

```
8 • CREATE TABLE staging_sales_data(  
9     transaction_id VARCHAR (50), # temporary hold "TXN-6867343"  
10    customer_id VARCHAR (50),  
11    category VARCHAR (100),  
12    item VARCHAR (100),  
13    price_per_unit VARCHAR (50),  
14    quantity VARCHAR (50),  
15    total_spent VARCHAR (50),  
16    payment_method VARCHAR(50),  
17    location VARCHAR(50),  
18    transaction_date VARCHAR(50),  
19    discount_applied VARCHAR(10)  
20 );  
21  
22 # Creating a production table  
23 • CREATE TABLE dim_sales_performance (  
24     transaction_id INT PRIMARY KEY,  
25     customer_id VARCHAR (20),  
26     category VARCHAR (100),  
27     total_spend DECIMAL (10,2),  
28     transaction_date DATE  
29 );
```

Step 2) Importing Dataset into Staging Sales Data Table

11 * `SELECT * FROM staging_sales_data;`

transaction_id	customer_id	category	item	price_per_unit	quantity	total_spent	payment_method	location	transaction_date	discount_applied
TWN_6957343	CUST_09	Patisserie	Item_10_PAT	18.5	10.0	185.0	Digital Wallet	Online	2024-04-08	True
TWN_3731988	CUST_22	Milk Products	Item_17_MLK	29.0	9.0	261.0	Digital Wallet	Online	2023-07-23	True
TWN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2.0	43.0	Credit Card	Online	2022-10-03	False
TWN_9458125	CUST_06	Beverages	Item_16_BEV	27.5	9.0	247.5	Credit Card	Online	2022-05-07	
TWN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7.0	87.5	Digital Wallet	Online	2022-10-02	False
TWN_7482416	CUST_09	Patisserie			10.0	200.0	Credit Card	Online	2023-11-30	
TWN_3612209	CUST_07	Food	Item_1_FOOD	5.0	8.0	40.0	Credit Card	In-store	2023-06-10	True
TWN_1372952	CUST_21	Furniture		33.5			Digital Wallet	In-store	2024-04-02	True
TWN_9729486	CUST_23	Furniture	Item_16_FUR	27.5	1.0	27.5	Credit Card	In-store	2023-04-25	False
TWN_3722661	CUST_25	Butchers	Item_22_BUT	36.5	3.0	109.5	Cash	Online	2024-03-14	False
TWN_8776416	CUST_22	Butchers	Item_3_BUT	8.0	9.0	72.0	Cash	In-store	2024-12-14	True
TWN_5422631	CUST_09	Milk Products			8.0	52.0	Digital Wallet	In-store	2023-01-12	True

12 * `SELECT COUNT(*) FROM staging_sales_data;`

13 *

14 *

15 *

16 *

17 *

18 *

19 *

20 *

21 *

22 *

23 *

24 *

25 *

26 *

27 *

28 *

29 *

30 *

31 *

32 *

33 *

34 *

35 *

36 *

37 *

38 *

39 *

40 *

41 *

42 *

43 *

44 *

45 *

46 *

47 *

48 *

49 *

50 *

51 *

52 *

53 *

54 *

55 *

56 *

57 *

58 *

59 *

60 *

61 *

62 *

63 *

64 *

65 *

66 *

67 *

68 *

69 *

70 *

71 *

72 *

73 *

74 *

75 *

76 *

77 *

78 *

79 *

80 *

81 *

82 *

83 *

84 *

85 *

86 *

87 *

88 *

89 *

90 *

91 *

92 *

93 *

94 *

95 *

96 *

97 *

98 *

99 *

100 *

101 *

102 *

103 *

104 *

105 *

106 *

107 *

108 *

109 *

110 *

111 *

112 *

113 *

114 *

115 *

116 *

117 *

118 *

119 *

120 *

121 *

122 *

123 *

124 *

125 *

126 *

127 *

128 *

129 *

130 *

131 *

132 *

133 *

134 *

135 *

136 *

137 *

138 *

139 *

140 *

141 *

142 *

143 *

144 *

145 *

146 *

147 *

148 *

149 *

150 *

151 *

152 *

153 *

154 *

155 *

156 *

157 *

158 *

159 *

160 *

161 *

162 *

163 *

164 *

165 *

166 *

167 *

168 *

169 *

170 *

171 *

172 *

173 *

174 *

175 *

176 *

177 *

178 *

179 *

180 *

181 *

182 *

183 *

184 *

185 *

186 *

187 *

188 *

189 *

190 *

191 *

192 *

193 *

194 *

195 *

196 *

197 *

198 *

199 *

200 *

201 *

202 *

203 *

204 *

205 *

206 *

207 *

208 *

209 *

210 *

211 *

212 *

213 *

214 *

215 *

216 *

217 *

218 *

219 *

220 *

221 *

222 *

223 *

224 *

225 *

226 *

227 *

228 *

229 *

230 *

231 *

232 *

233 *

234 *

235 *

236 *

237 *

238 *

239 *

240 *

241 *

242 *

243 *

244 *

245 *

246 *

247 *

248 *

249 *

250 *

251 *

252 *

253 *

254 *

255 *

256 *

257 *

258 *

259 *

260 *

261 *

262 *

263 *

264 *

265 *

266 *

267 *

268 *

269 *

270 *

271 *

272 *

273 *

274 *

275 *

276 *

277 *

278 *

279 *

280 *

281 *

282 *

283 *

284 *

285 *

286 *

287 *

288 *

289 *

290 *

291 *

292 *

293 *

294 *

295 *

296 *

297 *

298 *

299 *

300 *

301 *

302 *

303 *

304 *

305 *

306 *

307 *

308 *

309 *

310 *

311 *

312 *

313 *

314 *

315 *

316 *

317 *

318 *

319 *

320 *

321 *

322 *

323 *

324 *

325 *

326 *

327 *

328 *

329 *

330 *

331 *

332 *

333 *

334 *

335 *

336 *

337 *

338 *

339 *

340 *

341 *

342 *

343 *

344 *

345 *

346 *

347 *

348 *

349 *

350 *

351 *

352 *

353 *

354 *

355 *

356 *

357 *

358 *

359 *

360 *

361 *

362 *

363 *

364 *

365 *

366 *

367 *

368 *

369 *

370 *

371 *

372 *

373 *

374 *

375 *

376 *

377 *

378 *

379 *

380 *

381 *

382 *

383 *

384 *

385 *

386 *

387 *

388 *

389 *

390 *

391 *

392 *

393 *

394 *

395 *

396 *

397 *

398 *

399 *

400 *

401 *

402 *

403 *

404 *

405 *

406 *

407 *

408 *

409 *

410 *

411 *

412 *

413 *

414 *

415 *

416 *

417 *

418 *

419 *

420 *

421 *

422 *

423 *

424 *

425 *

426 *

427 *

428 *

429 *

430 *

431 *

432 *

433 *

434 *

435 *

436 *

437 *

438 *

439 *

440 *

441 *

442 *

443 *

444 *

445 *

446 *

447 *

448 *

449 *

450 *

451 *

452 *

453 *

454 *

455 *

456 *

457 *

458 *

459 *

460 *

461 *

462 *

463 *

464 *

465 *

466 *

467 *

468 *

469 *

470 *

471 *

472 *

473 *

474 *

475 *

476 *

477 *

478 *

479 *

480 *

481 *

482 *

483 *

484 *

485 *

486 *

487 *

488 *

489 *

490 *

491 *

492 *

493 *

494 *

495 *

496 *

497 *

498 *

499 *

500 *

501 *

502 *

503 *

504 *

505 *

506 *

507 *

508 *

509 *

510 *

511 *

512 *

513 *

514 *

515 *

516 *

517 *

518 *

519 *

520 *

521 *

522 *

523 *

524 *

525 *

526 *

527 *

528 *

529 *

530 *

531 *

532 *

533 *

534 *

535 *

536 *

537 *

538 *

539 *

540 *

541 *

542 *

543 *

544 *

545 *

546 *

547 *

548 *

549 *

550 *

551 *

552 *

553 *

554 *

555 *

556 *

557 *

558 *

559 *

560 *

561 *

562 *

563 *

564 *

565 *

566 *

567 *

568 *

569 *

570 *

571 *

572 *

573 *

574 *

575 *

576 *

577 *

578 *

579 *

580 *

581 *

582 *

583 *

584 *

585 *

586 *

587 *

588 *

589 *

590 *

591 *

592 *

593 *

594 *

595 *

596 *

597 *

598 *

599 *

600 *

601 *

602 *

603 *

604 *

605 *

606 *

607 *

608 *

609 *

610 *

611 *

612 *

613 *

614 *

615 *

616 *

617 *

618 *

619 *

620 *

621 *

622 *

623 *

624 *

625 *

626 *

627 *

628 *

629 *

630 *

631 *

632 *

633 *

634 *

635 *

636 *

637 *

638 *

639 *

640 *

641 *

642 *

643 *

644 *

645 *

646 *

647 *

648 *

649 *

650 *

651 *

652 *

653 *

654 *

655 *

656 *

657 *

658 *

659 *

660 *

661 *

662 *

663 *

664 *

665 *

666 *

667 *

668 *

669 *

670 *

671 *

672 *

673 *

674 *

675 *

676 *

677 *

678 *

679 *

680 *

681 *

682 *

683 *

684 *

685 *

686 *

687 *

688 *

689 *

690 *

691 *

692 *

693 *

694 *

695 *

696 *

697 *

698 *

699 *

700 *

701 *

702 *

703 *

704 *

705 *

706 *

707 *

708 *

709 *

710 *

711 *

712 *

713 *

714 *

715 *

716 *

717 *

718 *

719 *

720 *

721 *

722 *

723 *

724 *

725 *

726 *

727 *

728 *

729 *

730 *

731 *

732 *

733 *

734 *

735 *

736 *

737 *

738 *

739 *

740 *

741 *

742 *

743 *

744 *

745 *

746 *

747 *

748 *

749 *

750 *

751 *

752 *

753 *

754 *

755 *

756 *

757 *

758 *

759 *

760 *

761 *

762 *

763 *

764 *

765 *

766 *

767 *

768 *

769 *

770 *

771 *

772 *

773 *

774 *

775 *

776 *

777 *

778 *

779 *

780 *

781 *

782 *

783 *

784 *

785 *

786 *

787 *

788 *

789 *

790 *

791 *

792 *

793 *

794 *

795 *

796 *

797 *

798 *

799 *

800 *

801 *

802 *

803 *

804 *

805 *

806 *

807 *

808 *

809 *

810 *

811 *

812 *

813 *

814 *

815 *

816 *

817 *

818 *

819 *

820 *

821 *

822 *

823 *

824 *

825 *

826 *

827 *

828 *

829 *

830 *

831 *

832 *

833 *

834 *

835 *

836 *

837 *

838 *

839 *

840 *

841 *

842 *

843 *

844 *

845 *

846 *

847 *

848 *

849 *

850 *

851 *

852 *

853 *

854 *

855 *

856 *

857 *

858 *

859 *

860 *

861 *

862 *

863 *

864 *

865 *

866 *

867 *

868 *

869 *

870 *

871 *

872 *

873 *

874 *

875 *

876 *

877 *

878 *

879 *

880 *

881 *

882 *

883 *

884 *

885 *

886 *

887 *

888 *

889 *

890 *

891 *

892 *

893 *

894 *

895 *

896 *

897 *

898 *

899 *

900 *

901 *

902 *

903 *

904 *

905 *

906 *

907 *

908 *

909 *

910 *

911 *

912 *

913 *

914 *

915 *

916 *

917 *

918 *

919 *

920 *

921 *

922 *

923 *

924 *

925 *

926 *

927 *

928 *

929 *

930 *

931 *

932 *

933 *

934 *

935 *

936 *

937 *

938 *

939 *

940 *

941 *

942 *

943 *

944 *

945 *

946 *

947 *

948 *

949 *

950 *

951 *

952 *

953 *

954 *

955 *

956 *

957 *

958 *

959 *

960 *

961 *

962 *

963 *

964 *

965 *

966 *

967 *

968 *

969 *

970 *

971 *

972 *

973 *

974 *

975 *

976 *

977 *

978 *

979 *

980 *

981 *

982 *

983 *

984 *

985 *

986 *

987 *

988 *

989 *

990 *

991 *

992 *

993 *

994 *

995 *

996 *

997 *

998 *

999 *

1000 *

Step 3) ETL process (Moving & Transforming Data from staging to production)

- Transform text into number using CAST_AS UNSIGNED, to decimal (10,2) using CAST_AS DECIMAL (10,2), transform to date with STR_TO_DATE ('%Y-%m-%d')

```

35 #Moving & transforming data from staging to production
36 * INSERT INTO dim_sales_performance (
37     transaction_id,
38     customer_id,
39     category,
40     total_spent,
41     transaction_date
42 )
43 SELECT
44     #Transforming data & type into number & INT
45     CAST(REPLACE(transaction_id, 'TXN_', '') AS UNSIGNED),
46     customer_id,
47     category,
48     #Performing FORCE CLEAN for dirty dataset: If total_spent is empty, '0', or spaces, make it 0.00
49     CASE
50         WHEN TRIM(total_spent) = '' OR total_spent IS NULL THEN 0.00
51         WHEN total_spent = '0' THEN 0.00
52         ELSE CAST(total_spent AS DECIMAL(10,2))
53     END,
54     #Fixing the date format
55     STR_TO_DATE(transaction_date, '%Y-%m-%d')
56 FROM staging_sales_data;

```

Step – 4) EDA in Python

(4.1) Importing Pandas, Matplotlib, Numpy

```
#Importing the libraries
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
df = pd.read_csv("MNC_Projects/cleaned_mnc_retail.csv")
```

```
df.head()
```

	transaction_id	customer_id	category	quantity	price_per_unit	total_spend	transaction_date
0	1002182	CUST_01	Food	5	11	55.0	2024-10-08
1	1003865	CUST_15	Furniture	5	7	32.5	2022-03-12
2	1003940	CUST_06	Furniture	9	11	99.0	2022-04-22
3	1004091	CUST_04	Food	3	41	123.0	2023-11-09
4	1004124	CUST_08	Computers and electric accessories	5	14	70.0	2022-03-02

```
df = pd.read_csv("MNC_Projects/cleaned_mnc_retail.csv", sep=";")
df.head()
```

	transaction_id	customer_id	category	quantity	price_per_unit	total_spend	transaction_date
0	1002182	CUST_01	Food	5	11	55.0	2024-10-08
1	1003865	CUST_15	Furniture	5	7	32.5	2022-03-12
2	1003940	CUST_06	Furniture	9	11	99.0	2022-04-22
3	1004091	CUST_04	Food	3	41	123.0	2023-11-09
4	1004124	CUST_08	Computers and electric accessories	5	14	70.0	2022-03-02

```
df.columns = df.columns.str.replace(' ', '').str.strip()
df.head()
```

	transaction_id	customer_id	category	quantity	price_per_unit	total_spend	transaction_date
0	1002182	CUST_01	Food	5	11	55.0	2024-10-08
1	1003865	CUST_15	Furniture	5	7	32.5	2022-03-12
2	1003940	CUST_06	Furniture	9	11	99.0	2022-04-22
3	1004091	CUST_04	Food	3	41	123.0	2023-11-09
4	1004124	CUST_08	Computers and electric accessories	5	14	70.0	2022-03-02

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   transaction_id  12575 non-null  int64
 1   customer_id     12575 non-null  object
 2   category        12575 non-null  object
 3   quantity        12575 non-null  int64
 4   price_per_unit  12575 non-null  int64
 5   total_spend     12575 non-null  float64
 6   transaction_date 12575 non-null  object
dtypes: float64(1), int64(3), object(3)
memory usage: 687.8+ KB
```

```
#Changing Data Type
df["transaction_date"] = pd.to_datetime(df["transaction_date"])

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12575 entries, 0 to 12574
Data columns (total 7 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   transaction_id         12575 non-null  int64  
 1   customer_id            12575 non-null  object  
 2   category               12575 non-null  object  
 3   quantity               12575 non-null  int64  
 4   price_per_unit         12575 non-null  int64  
 5   total_spend            12575 non-null  float64 
 6   transaction_date       12575 non-null  datetime64[ns]
dtypes: datetime64[ns](1), float64(1), int64(3), object(2)
memory usage: 687.8+ KB

#Checking dataset size
df.shape

(12575, 7)

#Checking Missing Value
df.isnull().sum()

transaction_id    0
customer_id       0
category          0
quantity          0
price_per_unit    0
total_spend       0
transaction_date  0
dtype: int64
```

Step 5) Creating KPIs (Total Revenue, Total Transactions, Total Cus, Total Avg Cus)

```
#Creating KPI (total revenue)
total_revenue = df["total_spend"].sum()
total_revenue

1552071.0

total_revenue = df["total_spend"].sum()
print("Total Revenue:", round(total_revenue,2))

Total Revenue: 1552071.0

# Total Unique Transaction Count
total_transaction = df["transaction_id"].nunique()
total_transaction

12575

total_transaction = df["transaction_id"].nunique()
print("Total Transactions:", total_transaction)

Total Transactions: 12575

# Total Unique Customer Count
total_customer = df["customer_id"].nunique()
total_customer

25

total_customer = df["customer_id"].nunique()
print("Total Customers:", total_customer)

Total Customers: 25

# Average Order Value
aov = df["total_spend"].mean()
round(aov,2)
```

Step 6) Adding more columns for time intelligence analysis

```
#Creating new columns for time intelligence analysis
df["transaction_date"] = pd.to_datetime(df["transaction_date"])
```

```
df["year"] = df["transaction_date"].dt.year
df["month"] = df["transaction_date"].dt.month
df["day"] = df["transaction_date"].dt.day
```

```
df.head()
```

	transaction_id	customer_id	category	quantity	price_per_unit	total_spend	transaction_date	year	month	day
0	1002182	CUST_01	Food	5	11	55.0	2024-10-08	2024	10	8
1	1003865	CUST_15	Furniture	5	7	32.5	2022-03-12	2022	3	12
2	1003940	CUST_06	Furniture	9	11	99.0	2022-04-22	2022	4	22
3	1004091	CUST_04	Food	3	41	123.0	2023-11-09	2023	11	9
4	1004124	CUST_08	Computers and electric accessories	5	14	70.0	2022-03-02	2022	3	2

Step 7) KPIs Generation

Finding revenue by category and monthly revenue trends

```
#Revenue by Category
category_sales = df.groupby("category")["total_spend"].sum().sort_values(ascending=False)
category_sales
```

```
category
Butchers                208118.0
Electric household essentials  203813.5
Beverages               197047.5
Furniture               195310.0
Food                   194812.0
Computers and electric accessories  190692.5
Patisserie              182165.5
Milk Products          180112.0
Name: total_spend, dtype: float64
```

```
#Monthly Revenue
monthly_sales = df.groupby(["year", "month"])["total_spend"].sum()
monthly_sales
```

```
year  month
2022   1      52911.5
      2      43325.5
      3      40996.0
      4      40442.0
      5      40347.5
      6      42576.0
      7      44471.5
      8      41333.5
      9      46113.5
     10      38355.0
     11      41256.5
     12      38201.0
2023   1      48052.5
      2      39214.5
      3      38534.5
      4      38905.5
```

Finding Top Customers & Qty Sold per Category

```
#Finding Top Customers
top_customers = df.groupby("customer_id")["total_spend"].sum().sort_values(ascending = False).head(10)
top_customers

customer_id
CUST_24    68452.0
CUST_08    67351.5
CUST_05    66974.5
CUST_16    65570.5
CUST_13    65037.0
CUST_23    64507.0
CUST_10    63155.5
CUST_15    63117.5
CUST_21    62933.0
CUST_02    62046.5
Name: total_spend, dtype: float64

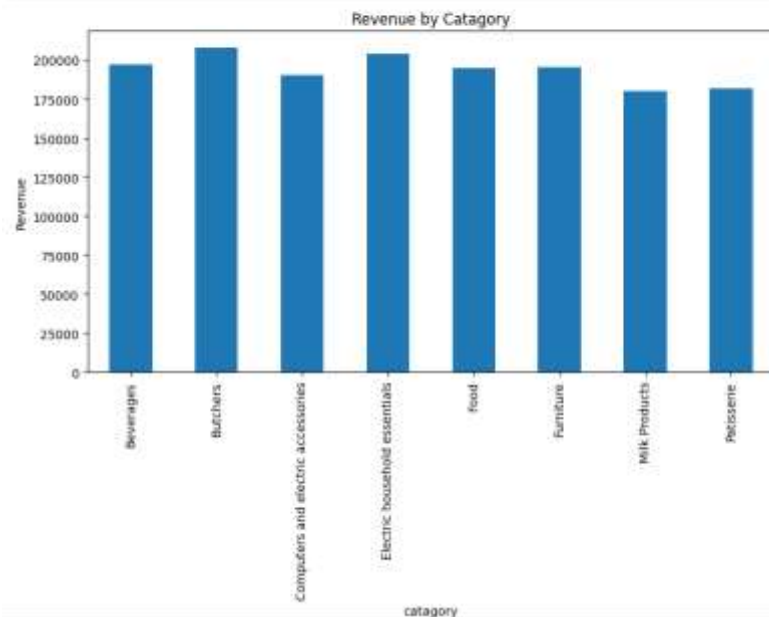
#Checking qty sold for category popularity analysis
category_qty_sold = df.groupby("category")["quantity"].sum().sort_values(ascending=False)
category_qty_sold

category
Furniture            8462
Food                 8387
Beverages            8358
Milk Products        8339
Electric household essentials  8309
Computers and electric accessories  8272
Butchers             8206
Patisserie           7943
Name: quantity, dtype: int64
```

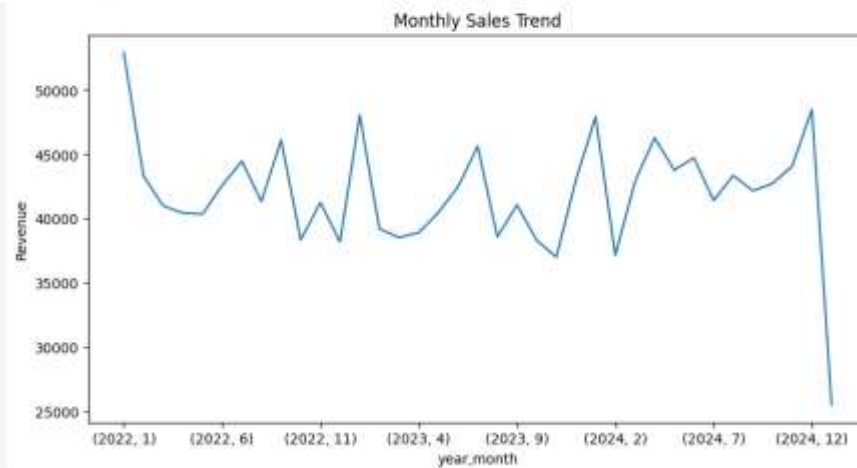
Step 8) Visualization

Creating a quick visualization for Category Value & Monthly Sales Trend

```
#Quick visualization for category value
category_sales = df.groupby("category")["total_spend"].sum()
category_sales.plot(kind="bar", figsize=(10,3))
plt.title("Revenue by Category")
plt.ylabel("Revenue")
plt.show()
```




```
#Creating Monthly Sales Trends Visualization
monthly_sales = df.groupby(["year","month"]).total_spent.sum()
monthly_sales.plot(figsize=(10,5))
plt.title("Monthly Sales Trend")
plt.ylabel("Revenue")
plt.show()
```



Step 9) Exporting & saving the cleaned file

```
#Exporting cleaned data from Python
df.to_csv("sales_data_clean.csv", index=False)
```

```
pd.read_csv("sales_data_clean.csv").head()
```

	transaction_id	customer_id	catagory	quantity	price_per_unit	total_spent	transaction_date	year	month	day
0	1002182	CUST_01	Food	5	11	55.0	2024-10-08	2024	10	8
1	1003865	CUST_15	Furniture	5	7	32.5	2022-03-12	2022	3	12
2	1003940	CUST_06	Furniture	9	11	99.0	2022-04-22	2022	4	22
3	1004091	CUST_04	Food	3	41	123.0	2023-11-09	2023	11	9
4	1004124	CUST_08	Computers and electric accessories	5	14	70.0	2022-03-02	2022	3	2

```
#saving the file in laptop location
df.to_csv(r"E:\Projects\MNC_Project_Portfolio_1\sales_data_clean.csv", index=False)
```

Step 9) Data Visualization in Power BI

Import data to power query editor, and inspect the entire dataset quality



Step 10) Creating one page visualization in power bi desktop, then upload to power bi service, then create a dashboard.



Final Result

